Interview Notes

Maninder (Kaurman) Kaur

August 16, 2026

Start

General Problem Solving

Always read the problem twice (ask clarifying questions. Try to think of different ways to solve a problem. Think end-to-end of the solutions based on complexity. Write the algorithm to form patterns in drawing. Code it out. Try and improve it once you think you've finished. Go through other solutions (even if answered correctly).

Arrays

217. Contains Duplicates

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

Example 1: Input: nums = [1,2,3,1] Output: true
Example 2: Input: nums = [1,2,3,4] Output: false
Example 3: Input: nums = [1,1,1,3,3,4,3,2,4,2] Output: true

```
# first attempt using two for loops
class Solution(object):
    def containsDuplicate(self, nums):
        """
        :type nums: List[int]
        :rtype: bool
        """
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True


        return False
# ran into time exceeded complication

# second attempt: use a set()
class Solution(object):
    def containsDuplicate(self, nums):
```

```
        """
        :type nums: List[int]
        :rtype: bool
        """
        numsdown = set(nums)

        if len(nums) != len(numsdown):
            return True

        return False
```

Notes: brute-forcing with for-loops will run into a time exceeded issue. With `set(nums)`, you can make a new array that removes duplicates and makes it an ordered list. If you notice the `len()` of the original and new arrays is different, you know a duplicate existed and was removed.

268. Missing Number

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Example 1: Input: nums = [3,0,1] Output: 2
Example 2: Input: nums = [0,1] Output: 2
Example 3: Input: nums = [9,6,4,2,3,5,7,0,1] Output: 8

```
# first attempt
class Solution(object):
    def missingNumber(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        newarr = sorted(nums)
        n = 0

        for i in range(len(newarr)):
            if i == newarr[i]:
                continue
            else:
                return i
            n = i

        return len(nums)
```

Notes: It's simply a list counting up and starts at 1. The `len(nums)` will give us the top number and then we know our range. Count up in a for loop and see if the array value equals i. If not, return. If loop is complete, simply return the `len(nums)` as you need to add the next value.

...

```
# better solution,
        return sum(range(len(nums)+1) - sum (nums)
```

Notes: This one first grabs the length of the nums list and adds 1. `range()` creates a list of numbers starting from index 0 to n-1 with values starting at 1, so it will correctly add all [0,n] numbers and we minus it by our `nums` which does not have the missing value.

448. Missing Disappeared Number

Given an array nums of n integers where nums[i] is in the range [1, n], return an array of all the integers in the range [1, n] that do not appear in nums.

Example 1: Input: nums = [4,3,2,7,8,2,3,1] Output: [5,6]

Example 2: Input: nums = [1,1] Output: [2]

```
# first try - brute force iterate, time exceeded
class Solution(object):
    def findDisappearedNumbers(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

        setnums = set(nums)
        diff = []

        for i in range(1, len(nums) + 1):
            if i not in set(nums):
                diff.append(i)

        return diff


# second try - in place check
class Solution(object):
    def findDisappearedNumbers(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

        for i in range(len(nums)):
            temp = abs(nums[i]) - 1
            if nums[temp] > 0:
                nums[temp] *= -1

        res = []
        for i,n in enumerate(nums):
            if n > 0:
                res.append(i+1)

        return res
```

Notes: In the very first one, we iterate through the set and count from 1 to n, and whichever isn't there, we append to a list and return the list. This runs into a time exceeded error.
However, there is a in place solution as well that works. We iterate through the range and reduce it by one value so the values are no longer [1,n] but [0,n-1] matching the indexes. Now, looking at the matching index, if it is positive, we make it negative, using it as a "seen" indicator. If at the second iteration, the value is still positive, then we know that index `i+1` is missing and we append it to `res[]`.

1. Two Sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

Example 1: Input: nums = [2,7,11,15], target = 9 Output: [0,1]
Example 2: Input: nums = [3,2,4], target = 6 Output: [1,2]
Example 3: Input: nums = [3,3], target = 6 Output: [0,1]

```
# first try - brute force with 2 for loops
class Solution(object):
    def twoSum(self, nums, target):
        """
        :type nums: List[int]
        :type target: int
        :rtype: List[int]
        """

        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i,j]

# second try - use a hash map
class Solution(object):
    def twoSum(self, nums, target):
        """
        :type nums: List[int]
        :type target: int
        :rtype: List[int]
        """

        hashm = {}

        for i, n in enumerate(nums):
            if target - n in hashm:
                return i, hashm[target-n]
            else:
                hashm[n] = i
```

Notes: At first, you could easily do a brute force method with 2 for loops, one that starts at 0, and one that is always one ahead and return the pair. But that takes $O(n^2)$ time.
But with a hash map, we can work in $O(n)$ time. We create a hashmap called `hashm`. We iterate with index and value, and see if whatever value we are at and the target minus that value is in the hashmap. If it it, we return to two. If not, we add a key with that value and have it equal to the index.

1365. How Numbers Are Smaller?

Given the array nums, for each nums[i] find out how many numbers in the array are smaller than it. That is, for each nums[i] you have to count the number of valid j's such that j ! = i and nums[j] < nums[i]. Return the answer in an array.
Example 1: Input: nums = [8,1,2,2,3] Output: [4,0,1,1,3]
Example 2: Input: nums = [6,5,4,8] Output: [2,1,0,3]
Example 3: Input: nums = [7,7,7,7] Output: [0,0,0,0]

```
# first try - sort and get value
class Solution(object):
    def smallerNumbersThanCurrent(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

        res = []
        sortnums = sorted(nums)

        for i,n in enumerate(nums):
            res.append(sortnums.index(n))

        return res
```

Notes: Make a new list for the small number counter. Then we sort the array, placing them in an index. The smallest number is already at 0, and if there are duplicates, it'll simply return the first occurrence of it. Then we traverse nums, and for each one, place the index from sorted into `res[]`

1266. Minimum Time Travel

On a 2D plane, there are n points with integer coordinates points[i] = [xi, yi]. Return the minimum time in seconds to visit all the points in the order given by points.

You can move according to these rules:

- In 1 second, you can either:
  - move vertically by one unit,
  - move horizontally by one unit, or
  - move diagonally sqrt(2) units (in other words, move one unit vertically then one unit horizontally in 1 second).
- You have to visit the points in the same order as they appear in the array.

- You are allowed to pass through points that appear later in the order, but these do not count as visits.

Example 1:
Input: points = [[1,1],[3,4],[-1,0]] Output: 7
Example 2: Input: points = [[3,2],[-2,2]] Output: 5

```
# first try - use arithmatic
class Solution(object):
    def minTimeToVisitAllPoints(self, points):
        """
        :type points: List[List[int]]
        :rtype: int
        """
        res = 0

        x1, y1 = points.pop()

        while points:
            x2, y2 = points.pop()
            res += max(abs(y2-y1), abs(x2-x1))
            x1, y1 = x2, y2

        return res
```

Notes: What the equation comes down to is what is the biggest difference in x or y values from one point to another? First, we make a counter `res` and grab the very first values from points by using `pop()`. While we still have points, we iterate between each point and find the max difference and add that to `res`, then proceed to make those new points `x1, y1`.

54. Spiral Matrix

Given an m x n matrix, return all elements of the matrix in spiral order.
Example 1: Input: matrix = [[1,2,3],[4,5,6],[7,8,9]] Output: [1,2,3,6,9,8,7,4,5]
Example 2 Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]] Output: [1,2,3,4,8,12,11,10,9,5,6,7]

```
class Solution(object):
    def spiralOrder(self, matrix):
        """
        :type matrix: List[List[int]]
        :rtype: List[int]
        """

        res = []

        while matrix:

            res += matrix.pop(0)

            if matrix and matrix[0]:
```

```
            for row in matrix:
                res.append(row.pop())

        if matrix:
            res += matrix.pop()[::-1]

        if matrix and matrix[0]:
            for row in matrix[::-1]:
                res.append(row.pop(0))

    return res
```

Notes: It's simple, you you have to return the top row, the last element of each row, the last row reversed, and the first element of each row reverse and keep going.
To do this we first initialize our list, `res[]`. Then, while `matrix` still has values, we do the following:

- We simply return the top row, no need to append, simply $+=$.
- Then make sure that we still have matrix left before we gather each `row` and `pop()` the very last element from each row.
- Reverse the last row, and append each element if there is more matrix that has not been explored.
- Finally, reverse the rows so the last one is up now, and append each `row`'s first value.
- Repeat for every non-visited value, taken account by `while matrix:`

200. Number of Islands

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.
Example 1: Input: grid =

[["1","1","1","1","0"],["1","1","0","1","0"],["1","1","0","0","0"],["0","0","0","0","0"]]

Output: 1
Example 2: Input: grid =

[["1","1","0","0","0"],["1","1","0","0","0"],["0","0","1","0","0"],["0","0","0","1","1"]]

Output: 3

```
from collections import deque

class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
```

```
        :rtype: int
        """

        if not grid:
            return 0

        def bfs(r,c):
            q = deque()
            visit.add((r,c))
            q.append((r,c))

            while q:
                row, col = q.popleft()
                directions = [[1,0], [-1, 0], [0, 1], [0, -1]]

                for dr, dc in directions:
                    nr,nc = row + dr, col + dc
                    if(nr in range(rows) and nc in range(cols) and grid[nr][nc] ==
                        q.append((nr,nc))
                        visit.add((nr,nc))

        count = 0
        rows = len(grid)
        cols = len(grid[0])
        visit = set()

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == "1" and (r,c) not in visit:
                    bfs(r,c)
                    count += 1

        return count
```

Notes: First, we make sure there even is a grid. Secondly, we make a couple of variables:

`count`: This counts how many islands there are

`rows` and `cols`: As the name suggests, this is the number of rows and columns in the grid.

`visit`: Simply a set we use to see if we have visited a coordinate.

After that, for each `r` and `c` in `grid` we see if it is equal to 1 and if we have already visited yet. If not, we run `bfs(r,c)` and add one island.
How does BFS work? First we create a double-ended queue called `q`. We also make the points `(r,c)` visited and append them into the queue. The while loops keeps processing as long as there are cells to process. Then we create two new variables `row, col` that holds the value from `(r,c)` while also initializing the directions (up, down, left, right) to see. For those 4 directions, if we find a match of 1 again in that index, we add it to `visit`.

Arrays with 2 Pointers

121. Best Time Stocks

You are given an array prices where prices[i] is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1: Input: prices = [7,1,5,3,6,4] Output: 5

Example 2: Input: prices = [7,6,4,3,1] Output: 0

```
# first try - time exceeded brute force
class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """

        profit = 0

        for i in range(len(prices)):
            for j in range(i+1, len(prices)):
                if prices[j] > prices[i] and prices[j] - prices[i] > profit:
                    profit = prices[j] - prices[i]


        return profit


# second try - arithmatic
class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """

        minm = 10001
        maxp = 0

        for p in prices:
            if p < minm:
                minm = p
            elif p - minm > maxp:
                maxp = p - minm

        return maxp
```

Notes: Initially, you can brute force it by starting at index 0 and another index one ahead and seeing the farthest profit margin. However, you run into a time exceeded error.
But we can simply store another index in part 2. In that one, we have a minimum value. If we find a minimum value from the start, we store it. If not the minimum, we can check if at that value, the difference is bigger than a previous profit. Remember: do some math first.

977. Squares of Sorted Array

Given an integer array nums sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1: Input: nums = [-4,-1,0,3,10] Output: [0,1,9,16,100]
Example 2: Input: nums = [-7,-3,2,3,11] Output: [4,9,9,49,121]

```
# successful first run
class Solution(object):
    def sortedSquares(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

        for i in range(len(nums)):
            nums[i] *= nums[i]

        nums.sort()
        return nums

# more optimized approuch
class Solution(object):
    def sortedSquares(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

        if nums[0] >= 0:
            return [n**2 for n in nums]

        m = len(nums)
        for i,n in enumerate(nums):
            if n >= 0:
                m = i
                break

        # A is positive nums, B is reversed negatives
        A,B = nums[m:] , [-1 * n for n in reversed(nums[:m])]

        def merge(A,B):
            a = b = 0
```

```
        ret = []

        while a < len(A) and b < len(B):
            if A[a] < B[b]:
                ret.append(A[a])
                a += 1
            else:
                ret.append(B[b])
                b += 1

        if a < len(A):
            ret.extend(A[a:])
        else:
            ret.extend(B[b:])

        return [n**2 for n in ret]

    return merge(A,B)
```

Notes: We simply multiply each value in `nums` by itself, squaring it. Then we use `sort()` to sort it (duh). Best case $O(n)$, worst case $O\log(n)$.
In the second example, the code is more optimized. Firstly, if `nums` is all positive or equal to 0, we simple square every value and call it a day. If not, then we do the following: We create `A, B`, where one holds the all the positive numbers from an index `m` we found, or holds all the negative values, reversed. We then traverse through the lists until we hit a dead end and place them in order of before their square computation. Finally, we square the list `ret[]` with all the pre-squared values in order so they'll automatically be in order from least to greatest after squared.

15. 3Sum

Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0. Notice that the solution set must not contain duplicate triplets.

Example 1: Input: nums = [-1,0,1,2,-1,-4] Output: [[-1,-1,2],[-1,0,1]]
Example 2: Input: nums = [0,1,1] Output: []
Example 3: Input: nums = [0,0,0] Output: [[0,0,0]]

```
class Solution(object):
    def threeSum(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """

        nums.sort()
        res = []

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
```

```
                continue

            l = i + 1
            r = len(nums) - 1

            while l < r:
                csum = nums[i] + nums[l] + nums[r]

                if csum > 0:
                    r -= 1
                elif csum < 0:
                    l += 1
                else:
                    res.append([nums[i], nums[l], nums[r]])
                    l += 1

                    while l < r and nums[l] == nums[l - 1]:
                        l += 1

        return res
```

Notes: We start by sorting the list so that it's easier for us to keep track of the pointers and instantiating a list to put our results in.
Then, we start a loop that only goes up to the third to last number, since we'll have space for the two pointers to go to the last two.
We then start by seeing if there are duplicates, and if not, we make `l` equal to the value after `i` and `r` equal to the last value in the list. While `l < r`:
We make `csum` the total sum of all the variables. If `csum > 0`, then we know that the `r` value is too big so we decrease. If `csum < 0`, then we know that the `l` value is too small so we increase it. Else, we append the list variables to `res` and increase `l` to start repeating the process until `i`'s round isn't over.

3. Longest Substring Without Repeating Characters

Given a string s, find the length of the longest without duplicate characters.
Example 1: Input: s = "abcabcbb" Output: 3
Example 2: Input: s = "bbbbb" Output: 1
Example 3: Input: s = "pwwkew" Output: 3

```
class Solution(object):
    def lengthOfLongestSubstring(self, s):
        """
        :type s: str
        :rtype: int
        """

        hashm = {}
        count = 0
        left = 0
```

```
        for i, n in enumerate(s):
            if n in hashm and hashm[n] >= left:
                left = hashm[n] + 1

            hashm[n] = i

            curr = i - left + 1
            count = max(count, curr)

        return count
```

Notes: You could make a hash map that remembers each index and value and compares segment by segment. You first create a hashmap called `hashm` and two variables. `count` which keeps track of our final return value without repetition, and `left` which is a left boundary that keeps track from what starting point we are comparing the segment.
We traverse, keeping track of the index and value of the string given. If the value is in our hashmap and also in the segment boundary, we make the value afterwards the new left boundary.
Regardless of if inside the hashmap or not, we do make the value's index in the hashmap the index in the string.
Lastly, we see if the current value index minus the left boundary + 1 (since indexes start at 0) is bigger or not than the old counter we made in the beginning.

845. Longest Mountain in Array

You may recall that an array arr is a mountain array if and only if: arr.length ¿= 3 There exists some index i (0-indexed) with 0 < i < arr.length - 1 such that: arr[0] < arr[1] < ... < arr[i - 1] < arr[i] arr[i] > arr[i + 1] > ... > arr[arr.length - 1]

Given an integer array arr, return the length of the longest subarray, which is a mountain. Return 0 if there is no mountain subarray.

Example 1: Input: arr = [2,1,4,7,3,2,5] Output: 5 Explanation: The largest mountain is [1,4,7,3,2] which has length 5.

Example 2: Input: arr = [2,2,2] Output: 0 Explanation: There is no mountain.

```
class Solution(object):
    def longestMountain(self, arr):
        """
        :type arr: List[int]
        :rtype: int
        """

        ret = 0

        for i in range(1, len(arr)-1):
            if arr[i-1] < arr[i] > arr[i+1]:
                l = i
                r = i

                while l > 0 and arr[l] > arr[l-1]:
```

```
                l -= 1

            while r < len(arr)-1 and arr[r] > arr[r + 1]:
                r += 1

            ret = max (ret , r-l+1)

        return ret
```

Notes: We need to have two pointers. We start with a variable for the max amount in the mountains called `ret`. We then traverse from index 1 to the last item in `arr`. Now we have to see if the values before and after the value in `i` is less than it. If so, we start `l, r` in the index.
We will then traverse down from `i` using `l` and count how many values are smaller. Then we do the same with `r`. We take not of the index of the smallest value and the largest small value. We return the max: is it `ret` that we already have or the next boundary difference plus 1?

Arrays with Sliding Windows

219. Contains Duplicates II

Given an integer array nums and an integer k, return true if there are two distinct indices i and j in the array such that nums[i] == nums[j] and abs(i - j) <= k.
Example 1: Input: nums = [1,2,3,1], k = 3 Output: true
Example 2 Input: nums = [1,0,1,1], k = 1 Output: true
Example 3: Input: nums = [1,2,3,1,2,3], k = 2 Output: false

```
# slow O(n^2) solution
class Solution(object):
    def containsNearbyDuplicate(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
        :rtype: bool
        """

        check = False
        for i in range(0, len(nums)):
            for j in range(i+1, len(nums)):
                if nums[i] == nums[j] and abs(i-j) <= k:
                        check = True

        return check

# second attempt - sliding window
class Solution(object):
    def containsNearbyDuplicate(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
```

```
        :rtype: bool
        """

        seen = set()

        for i, n in enumerate(nums):
            if n in seen:
                return True
            seen.add(n)
            if len(seen) > k:
                seen.remove(nums[i-k])

        return False
```

Notes: We start with a set called `seen` which can only hold up to `k` amount of values. If there is a match, it cannot be returned if it doesn't also satisfy the absolute value condition as well.
Then, it traverse through the value and indices in `nums`. If a duplicate exists within the span of `k` in `seen`, we return true!
Otherwise, we add the latest value and if `seen` is more than `k`, which does not satisfy the requirement, we remove a value. If no duplicates are found, we just return false.